Understanding MRF-Style Image Segmentation

A line-by-line walkthrough of a simple OpenCV + NumPy segmentation script

---

This guide explains, step by step, how a simple Markov Random Field (MRF) style foreground/background image segmentation script works using OpenCV and NumPy. The code takes an image, makes an initial guess about which pixels belong to the foreground, and then iteratively refines that guess by looking at each pixel's four neighbours (top, bottom, left, right).

Full Source Code

```
import cv2
import numpy as np

# Read image
img = cv2.imread("image.jpg")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Create initial labels
# 0 = Background
# 1 = Foreground
labels = np.zeros(gray.shape, dtype=np.uint8)

# Simple initial classification
# Bright pixels = foreground
labels[gray > 150] = 1

# MRF-like neighbour checking
for iteration in range(5):

    new_labels = labels.copy()

    for i in range(1, gray.shape[0] - 1):
        for j in range(1, gray.shape[1] - 1):

            # Current pixel
            pixel = gray[i, j]

            # Four neighbours
            top = labels[i-1, j]
            bottom = labels[i+1, j]
            left = labels[i, j-1]
            right = labels[i, j+1]
            # Count foreground neighbours
```

```
            foreground_count = (
                top + bottom + left + right
            )

            # Decide label
            if foreground_count >= 3:
                new_labels[i, j] = 1
            elif foreground_count <= 1:
                new_labels[i, j] = 0

    labels = new_labels

# Convert labels into image
result = labels * 255

cv2.imshow("Original", img)
cv2.imshow("MRF Segmentation", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

1. Import OpenCV

```
import cv2
```

cv2 is the OpenCV library.

We use it to:

• read images
• convert images
• display images
• perform computer vision operations

Think: cv2 = tools for working with images

2. Import NumPy

```
import numpy as np
```

An image is basically a matrix of numbers. NumPy helps us work with these numbers. For example, a grayscale image may look like:

```
20   25   30   25   20
25  200  210  205   25
30  210  220  215   30
25  205  215  210   25
20   25   30   25   20
```

Each number represents the brightness of one pixel.

3. Read the Image

```
img = cv2.imread("image.jpg")
```

OpenCV loads **image.jpg**. The image is stored inside **img**. Initially, the image contains colour information. For example, one pixel might contain:

```
B = 20
G = 40
R = 200
```

4. Convert Image to Grayscale

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

This converts the colour image into a grayscale image. Instead of B G R, we now have only brightness:

```
0   -> black
50  -> dark
150 -> medium bright
255 -> white
```

So now we can easily compare pixels.

5. Create an Initial Label Image

```
labels = np.zeros(gray.shape, dtype=np.uint8)
```

This is very important. We create another matrix called **labels**. Initially, every pixel is given **0**.

Remember: 0 = Background, 1 = Foreground.

Suppose our image is 5 × 5. Initially:

```
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

So initially we are saying: *"Everything is background."*

6. Initial Foreground Detection

```
labels[gray > 150] = 1
```

This is our first rough guess. It means: if a pixel has brightness greater than 150, call it foreground. For example, brightness image:

```
20   20   20   20   20
20  200  210  205   20
20  210  220  215   20
20  205  215  210   20
20   20   20   20   20
```

Pixels above 150 become 1. So labels become:

```
0 0 0 0 0
0 1 1 1 0
0 1 1 1 0
0 1 1 1 0
0 0 0 0 0
```

Here: 0 = Background, 1 = Foreground. This is only the initial guess.

7. Start Iterations

```
for iteration in range(5):
```

This means: repeat the neighbour-checking process 5 times. Why? Because MRF-type methods use information from neighbouring pixels to improve the segmentation.

```
Initial guess
      |
Check neighbours
      |
Improve labels
      |
Check neighbours again
      |
Improve again
      |
Repeat
```

8. Make a Copy

```
new_labels = labels.copy()
```

We make a copy of the current labels. Why? Because we want to calculate the new result separately.

We don't want changes to one pixel to immediately disturb calculations for other pixels during the same pass.

```
OLD labels
     |
Check pixels
     |
NEW labels
```

9. Go Through Image Rows

```
for i in range(1, gray.shape[0] - 1):
```

i represents the row number. For example: row 0, row 1, row 2, row 3, row 4.

We start at 1 and stop before the last row. Why? Because we need to look at top, bottom, left, right. The first row doesn't have a top neighbour. The last row doesn't have a bottom neighbour. So we avoid the boundary pixels.

10. Go Through Columns

```
for j in range(1, gray.shape[1] - 1):
```

j represents the column number. So: i = row, j = column. Together, **gray[i, j]** means: give me the pixel at row i, column j.

11. Get Current Pixel

```
pixel = gray[i, j]
```

This gets the brightness of the current pixel. For example, if we are looking at the centre value 210, then pixel = 210.

```
        100
         |
80   90  200  90  80
80  200  220 200  80
80  200  210 200  80
80   90  200  90  80
```

Important: In this simplified code, **pixel** is actually not used later. It is there to show that we can inspect the current pixel's intensity.

12. Get Top Neighbour

```
top = labels[i-1, j]
```

Suppose our current pixel is X, surrounded by TOP, BOTTOM, LEFT and RIGHT. Then **labels[i-1, j]** means: go one row upward. So this gives the top neighbour's label.

```
        TOP
         |
LEFT ->  X  <- RIGHT
         |
      BOTTOM
```

13. Get Bottom Neighbour

```
bottom = labels[i+1, j]
```

This means: go one row downward. So we get the bottom neighbour.

14. Get Left Neighbour

```
left = labels[i, j-1]
```

This means: go one column to the left.

15. Get Right Neighbour

```
right = labels[i, j+1]
```

This means: go one column to the right.

16. Now We Have Four Neighbours

For one pixel, suppose all four neighbours are foreground:

```
              TOP
               |
              [1]

LEFT [1] ->  [X]  <- RIGHT [1]

              [1]
               |
            BOTTOM
```

Suppose: top = 1, bottom = 1, left = 1, right = 1. All four neighbours are foreground.

17. Count Foreground Neighbours

```
foreground_count = (
    top + bottom + left + right
)
```

Since top = 1, bottom = 1, left = 1, right = 1, we calculate 1 + 1 + 1 + 1 = 4. Therefore, foreground_count = 4.

18. Decision Rule

```
if foreground_count >= 3:
    new_labels[i, j] = 1
```

This says: if at least 3 of the 4 neighbours are foreground, make the current pixel foreground. For example:

```
       1
       |
   1 -> X <- 1
       |
       0
```

We have 1 + 1 + 1 + 0 = 3. Therefore, X = 1. The pixel becomes foreground. This is the basic neighbour-smoothing idea.

19. Background Decision

```
elif foreground_count <= 1:
    new_labels[i, j] = 0
```

This says: if 0 or 1 neighbours are foreground, make the current pixel background.

```
       0
       |
   0 -> X <- 1
       |
```

```
0
```

Foreground count: 0 + 0 + 1 + 0 = 1. Therefore, X = 0.

20. What If Count = 2?

```
if foreground_count >= 3:
    ...
elif foreground_count <= 1:
    ...
```

Notice the if/elif structure below. What happens when foreground_count = 2? Nothing happens — the previous label is retained because new_labels[i, j] was initialized as a copy of labels.

```
count = 4  ->  foreground
count = 3  ->  foreground
count = 2  ->  keep previous decision
count = 1  ->  background
count = 0  ->  background
```

21. Update the Labels

```
labels = new_labels
```

After checking all pixels, we replace the old labels with the improved labels. Then the next iteration starts.

```
OLD LABELS
    |
Neighbour checking
    |
NEW LABELS
    |
Become current labels
```

22. Why Repeat 5 Times?

```
for iteration in range(5):
```

Imagine initially some pixels may be incorrectly labelled:

```
0 0 0 0 0
0 1 0 1 0
0 1 1 1 0
0 0 1 0 0
0 0 0 0 0
```

Neighbour checking can make the result more consistent:

```
0 0 0 0 0
0 1 1 1 0
0 1 1 1 0
0 1 1 1 0
0 0 0 0 0
```

Repeating the process can smooth isolated mistakes.

23. Convert Labels to a Visible Image

```
result = labels * 255
```

Remember: labels use 0 = background, 1 = foreground. But if we display this directly, 1 is almost black. So we multiply by 255. Therefore: 0 × 255 = 0, 1 × 255 = 255. So 0 → black, 1 → white.

```
0 0 0 0 0              0   0   0   0   0
0 1 1 1 0              0 255 255 255   0
0 1 1 1 0     ->       0 255 255 255   0
0 1 1 1 0              0 255 255 255   0
0 0 0 0 0              0   0   0   0   0
```

When displayed, this becomes a black background with a white foreground block.

24. Display Original Image

```
cv2.imshow("Original", img)
```

This opens a window showing the original image.

25. Display Segmentation

```
cv2.imshow("MRF Segmentation", result)
```

This shows the final black-and-white segmentation. BLACK = Background, WHITE = Foreground.

26. Wait for Key

```
cv2.waitKey(0)
```

This means: keep the image window open until a key is pressed.

27. Close Windows

```
cv2.destroyAllWindows()
```

Closes all OpenCV windows.

Summary: This script performs a very simple, majority-vote style relaxation (inspired by Markov Random Fields) to clean up a rough brightness-based foreground mask, by iteratively letting each pixel's label be influenced by its four immediate neighbours.